

PyOMNeT-AI: Enabling Efficient OMNeT++ Network Optimization Through Low-Overhead, Multi-Pattern Python AI Integration

Moustafa Roshdi^{*†}, Arindam Chackraborty^{*}, Amir Amri^{*}, Sahana Raghunandan^{*} and Reinhard German[†]

^{*}Fraunhofer Institute for Integrated Circuits IIS, Erlangen, Germany

[†]Friedrich-Alexander University (FAU), Erlangen-Nuremberg, Germany

Corresponding Author: moustafa.roshdi@iis.fraunhofer.de

Abstract—Integrating AI/ML into OMNeT++ based network simulations is increasingly important for studying and optimizing adaptive network behaviors under realistic discrete-event dynamics. A prominent example is 5G-Advanced and emerging 6G RAN optimization, where learned policies can support scheduling, mobility management, and resource control. However, integrating Python-native AI/ML workflows with OMNeT++ remains challenging: C++-only integrations limit model agility, while socket-based RL coupling introduces substantial serialization and process coordination overhead. This paper presents PyOMNeT-AI, the first comprehensive framework for Python-native AI integration in OMNeT++, providing three complementary patterns under unified design principles. PyCoSim enables bidirectional co-simulation with Python modules as first-class simulation components for message-driven AI logic. PyEmbed provides scoped interpreter embedding for efficient inference-centric workloads, achieving 0.28 μ s baseline overhead and sustaining over 3 million calls per second. ShmGymBridge offers high-throughput shared-memory coupling for Gym-compatible RL training, providing 2–6 $\times$ speedup over socket-based alternatives depending on workload and payload size. We present the design rationale, performance characterization across integration patterns, and selection guidelines validated through Simu5G-based RAN optimization studies, with applicability to broader OMNeT++ domains including IoT, vehicular networks, and smart grids.

Index Terms—AI interface, Python, OMNeT++, 6G-RAN, AI-Native RAN, AI/ML

I. INTRODUCTION

ARTIFICIAL intelligence / machine learning (AI/ML) - driven control is becoming a central tool for optimizing modern communication networks, where decision making increasingly relies on high-dimensional measurements, heterogeneous traffic, and rapidly varying radio and mobility conditions [1]. In this setting, network simulation remains essential, as it enables controlled experiments, repeatability, and the ability to explore “what-if” scenarios that are difficult or impractical to evaluate in live deployments. However, the AI/ML toolchain is overwhelmingly Python-centric—driven by widely used libraries and frameworks such as scikit-learn and PyTorch—while many high-fidelity network simulators are implemented as standalone C/C++ discrete-event engines (e.g., OMNeT++ and ns-3) [1, 2]. Consequently, enabling AI-driven networking studies requires reliable bridging mechanisms that connect Python training/inference workflows to

discrete-event simulation execution, combining rapid model iteration with simulator-grade fidelity.

A prominent example is the optimization of radio access networks (RAN) for 5G-Advanced and emerging 6G systems, spanning tasks such as scheduling, resource allocation, mobility management, and adaptive parameter tuning under diverse channel and traffic conditions. These tasks are increasingly studied using data-driven approaches, including reinforcement learning (RL) for closed-loop decision making [3]. The complexity of such systems motivates the use of simulation frameworks capable of modeling protocol stacks and control procedures with sufficient detail [4, 5]. OMNeT++ is one such widely used discrete-event simulator with implementation of different air interface standards such as Simu5G which implements the 5G NR stack widely used for RAN research [5].

Despite this relevance, integrating Python-native AI workflows into OMNeT++ simulations remains cumbersome in practice. A common approach is to embed inference libraries directly into C++ simulation models [6]. Although effective for deploying fixed, offline-trained models, this often constrains the ML toolchain and reduces agility when models and pipelines evolve [2]. At the other end of the spectrum, many RL workflows depend on the OpenAI Gym interface and modern Python training libraries (e.g., Stable-Baselines), which assume an agent-driven interaction pattern via repeated environment steps. Connecting OMNeT++ to Gym through socket-based inter-process communication is feasible [7], but introduces serialization overhead and process coordination complexity that limits performance for high-frequency decisions or large observation spaces [8].

This paper introduces PyOMNeT-AI, a framework providing three complementary integration patterns under unified design principles for integrating Python ML/RL models into OMNeT++ simulations while preserving discrete-event semantics and enabling practical end-to-end workflows. PyOMNeT-AI is motivated by AI-driven RAN optimization in Simu5G-based simulations for 5G/6G research, but its design targets OMNeT++ use cases more broadly. The framework comprises three patterns that cover distinct points in the design space:

- **PyCoSim**: a binding-based co-simulation approach that enables bidirectional interaction between OMNeT++ and

Python, allowing Python modules to act as first-class simulation components while directly accessing OMNeT++ primitives.

- **PyEmbed:** a lightweight pattern that embeds a scoped Python interpreter only within selected AI-enabled modules, reducing build and iteration overhead while supporting rich Python-based inference and model logic.
- **ShmGymBridge:** a high-throughput shared-memory interface that couples OMNeT++ with Gym-compatible RL toolchains using synchronized observation/action exchange, avoiding socket and serialization bottlenecks while retaining the agent-driven stepping paradigm expected by modern RL libraries.

By presenting these patterns under a unified design rationale and taxonomy, PyOMNeT-AI enables practitioners to choose the integration mechanism that matches their model type, performance constraints, and development workflow. The concepts and interfaces described here are intended to be reusable; future users can instantiate their own optimization problems in OMNeT++ and implement the corresponding Python model or Gym environment wrapper.

The remainder of the paper is organized as follows: Section II reviews related work on AI/ML integration in OMNeT++ and ns-3. Section III presents PyOMNeT-AI’s design goals, umbrella architecture, and the three integration patterns. Section IV provides performance characterization and integration examples, and Section V concludes the paper.

II. BACKGROUND AND RELATED WORK

A common way to incorporate AI/ML in OMNeT++ is to embed ML inference directly into C++ simulation modules, leveraging native libraries or C++ APIs [2]. The omnetpp-ml repository [6] catalogs approaches including the TensorFlow C++ API and lightweight inference libraries such as frugally-deep for deploying Keras models in C++ simulations. While effective for deploying fixed models, these approaches constrain ML toolchains and impede rapid experimentation which are offered when using Python-based libraries and frameworks.

For reinforcement learning, the OpenAI Gym interface (now Gymnasium [9]) provides a standardized agent-environment API for RL research [10]. In the Gym API, the *agent* is the driver as it decides when to start an episode, when to apply each action, and when to reset. The *environment* is reactive and authoritative over the dynamics, returning the next observation and reward in response to the agent’s actions. To make discrete-event network simulators usable within this paradigm, a common approach runs the simulator and Python agent as separate processes connected through inter process communication (IPC). This socket-based architecture was first popularized in ns3-gym [11], which couples the Gym agent with ns-3 via socket communication. A similar approach was adopted for OMNeT++ through Veins-Gym [7], which exports Veins/OMNeT++ simulations as Gym environments using socket-based communication between Python agent wrappers and the OMNeT++ simulation environment.

TABLE I: Related AI Integration Approaches for Network Simulators

Approach	Integration	Overhead	RL Support	Platform
C++ Embedding [6]	In-process	Low	Limited	OMNeT++/ns-3
ns3-gym [11]	Socket IPC	High	Full	ns-3
Veins-Gym [7]	Socket IPC	High	Full	OMNeT++
ns3-ai [8]	Shared memory	Low	Full	ns-3
PyOMNeT-AI	Hybrid	Low	Full	OMNeT++

To address socket serialization overhead, ns3-ai [8] introduced shared-memory inter-process communication for connecting ns-3 with Python-based AI frameworks, demonstrating 10–100× speedup compared to ns3-gym. While shared memory reduces IPC overhead, it requires careful synchronization and memory management. Despite being very useful for RL-Gym-bases usecases, it might not be optimal for simple inference models. Additionally, to our knowledge, no equivalent comprehensive shared-memory RL integration has been developed for OMNeT++.

Table I summarizes these approaches. While each addresses specific integration needs, no existing framework provides a unified design covering the full spectrum from lightweight inference embedding to high-throughput RL training for OMNeT++. PyOMNeT-AI fills this gap by offering three complementary patterns under consistent design principles: PyCoSim for message-driven AI components, PyEmbed for efficient inference, and ShmGymBridge for RL training, as detailed in Section III.

III. PYOMNET-AI DESIGN AND INTEGRATION PATTERNS

A. Design Goals and Overall Architecture

AI-in-the-loop networking does not require a single fixed coupling style. Some use cases need Python logic to participate deeply in simulation behavior (e.g., message-driven control components), while others only need lightweight Python callbacks inside selected OMNeT++ modules (e.g., inference invoked at specific events). Reinforcement learning adds yet another interaction requirement, where most RL tooling assumes an agent-driven step loop consistent with the Gym/Gymnasium API—structurally different from event-driven discrete-event simulation. Because these regimes differ in control flow, latency requirements, and development overhead, PyOMNeT-AI provides three complementary integration patterns rather than forcing a single mechanism.

As shown in Fig. 1, PyOMNeT-AI offers three integration patterns, each positioned differently across the Python realm, the OMNeT++ realm, and a shared boundary where the interoperability mechanisms reside. The key idea is that Python-native AI can be connected to OMNeT++ at different depths depending on the required control flow and performance constraints, while keeping a consistent principle: OMNeT++ remains the discrete-event timing authority, and Python is integrated through explicit, well-defined interfaces. Performance characteristics of each pattern are detailed in Section IV. Across all three patterns, PyOMNeT-AI follows consistent design principles:

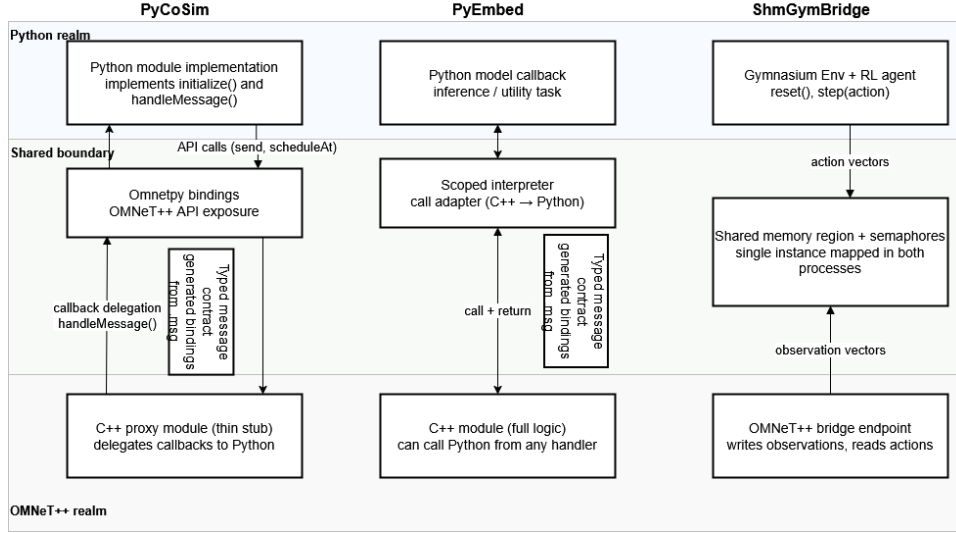


Fig. 1: PyOMNeT-AI umbrella architecture: realms, control flow, and data exchange.

- **Synchronous execution aligned with discrete-event semantics:** Whenever Python is invoked, it executes synchronously with the OMNeT++ event handler that triggered it, ensuring that simulation progression remains well-defined and tied to discrete-event processing.
- **Typed data exchange without external serialization in the in-process modes:** PyCoSim and PyEmbed are built around strongly typed exchanges that mirror OMNeT++ message semantics, avoiding ad-hoc external serialization formats for internal integration. This preserves message structure and reduces integration boilerplate when adding new message types.
- **Unified build workflow that produces both OMNeT++ and Python-facing artifacts:** The integration patterns rely on an explicit build process that generates and compiles the necessary bindings and modules so that Python code can access simulator-facing data types and interfaces.
- **Low overhead appropriate to the intended use case:** PyEmbed prioritizes minimal integration footprint for inference-centric workloads; ShmGymBridge prioritizes efficient step-level IPC for RL toolchains; PyCoSim prioritizes expressiveness and deep interaction for message-driven simulation components.

In the following subsections we explain in more detail the build and development flow of each integration pattern and how these design goals and principles are achieved.

B. PyCoSim: Python Modules as Simulation Components

PyCoSim is designed for scenarios where the AI logic should behave like a native part of the discrete-event model rather than an external helper. The defining principle is that the module behavior is implemented in Python, while OMNeT++ provides the event scheduling, message delivery, and simulation-time semantics. Practically, a lightweight C++

proxy module acts as the integration anchor inside OMNeT++, but delegates the core lifecycle and message-processing behavior to a Python class. For example, a gNB scheduler implemented in Python can receive messages, make allocation decisions, and emit scheduling grants—all while OMNeT++ manages event timing and message delivery. This is enabled by utilizing Omnetpy [12] as a build- and runtime dependency. Omnetpy also offers python bindings for C++ APIs and OMNeT++ classes, allowing users to subclass `cSimpleModule` from Python and access many other OMNeT++ bindings.

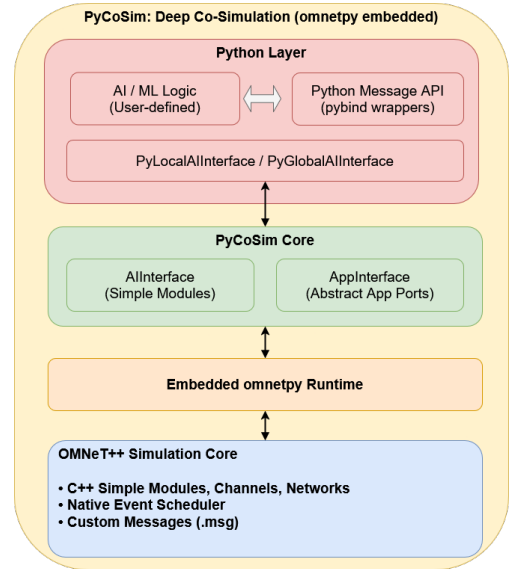


Fig. 2: PyCoSim Architecture overview and main components.

As shown in Fig. 2, PyCoSim’s architecture comprises four key elements: lightweight `AllInterface` adapters (`GlobalAllInterface` for centralized control coordinating multiple simulation entities such as a gNB-side controller, or `LocalAllInterface`

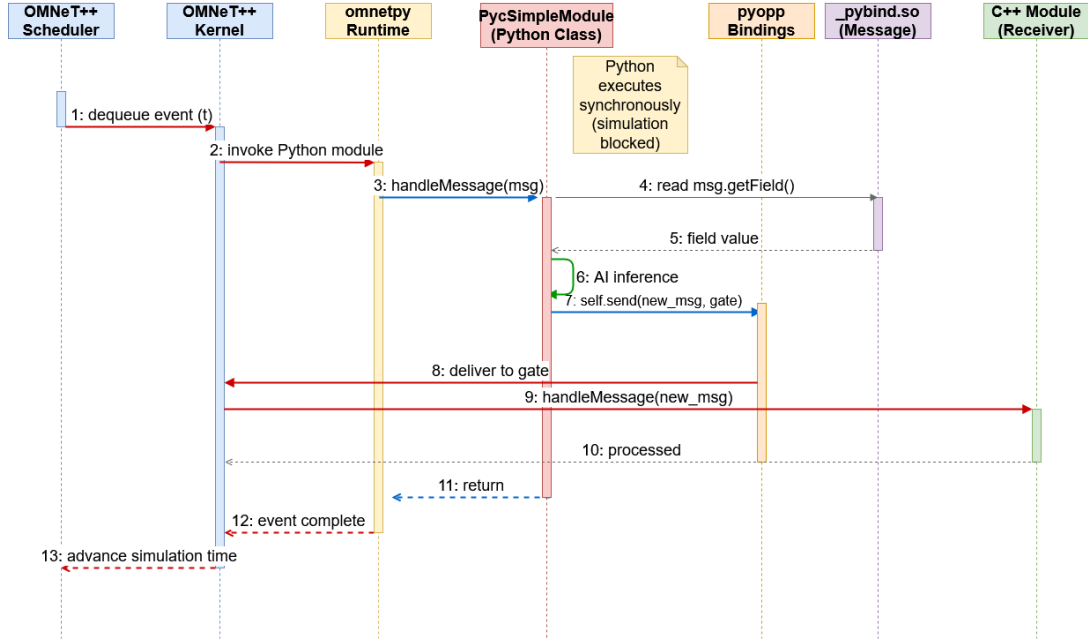


Fig. 3: PyCoSim Execution sequence.

for per-entity instances such as individual UEs), Python module classes implementing event handlers that behave like OMNeT++ modules, Pybind11 message bindings exposing OMNeT++ .msg types to Python, and AppInterface ports connecting applications to AI behavior while keeping application logic independent of the Python implementation.

The execution flow of PyCoSim is synchronous as shown in Fig. 3. The Python class provides module lifecycle logic such as initialize(), and the main event handler handleMessage(msg) is executed when the OMNeT++ scheduler delivers an event to the proxy C++ module. When an OMNeT++ event triggers a Python module call via the embedded runtime, Python reads fields via bindings, performs inference/processing, and sends a new OMNeT++ message. Only after Python returns does the event complete and the simulation advance. This synchronous execution ensures deterministic simulation while enabling full Python expressiveness.

From a user perspective, integrating a new PyCoSim-driven AI component typically follows these steps:

- Define OMNeT++ .msg types and expose them through PyCoSim bindings.
- Choose GlobalAIInterface (centralized) or LocalAIInterface (per-entity) and connect it in the NED model.
- Implement the AI behavior entirely in Python using the provided Python module templates/classes and connect applications to AI through the AppInterface ports.

C. PyEmbed: Lightweight Python Integration via a Scoped Interpreter

PyEmbed is a lightweight Python extension framework for OMNeT++ that avoids embedding a full co-simulation runtime (i.e., no omnetpy dependency) and instead integrates Python through pybind11 bindings plus a scoped interpreter

instantiated only inside selected modules. This approach minimizes integration footprint while maintaining native OMNeT++ event handling, ideal for inference-only workloads where full co-simulation is unnecessary.

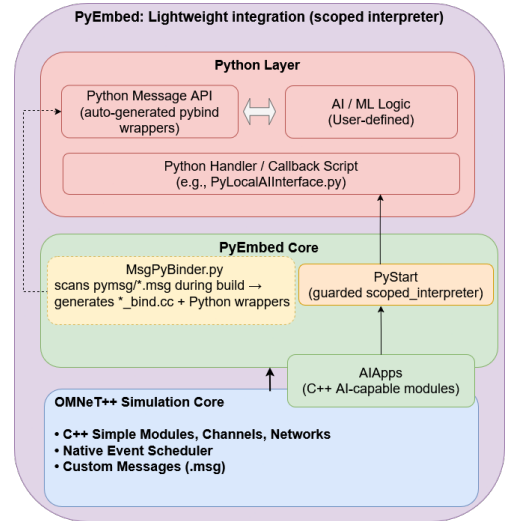


Fig. 4: PyEmbed Architecture overview and main components.

Fig. 4 shows PyEmbed’s architecture, which consists of Python-aware AIApps modules that load Python code/models at runtime, PyStart for scoped interpreter lifecycle management including safety/GIL handling (instantiated only when needed), and an automated binding generator (MsgPyBinder.py) that scans .msg files under the pymsg/ folder and produces Pybind11 bindings, eliminating manual binding code and keeping definitions consistent.

As for the execution sequence shown in Fig. 5, when the

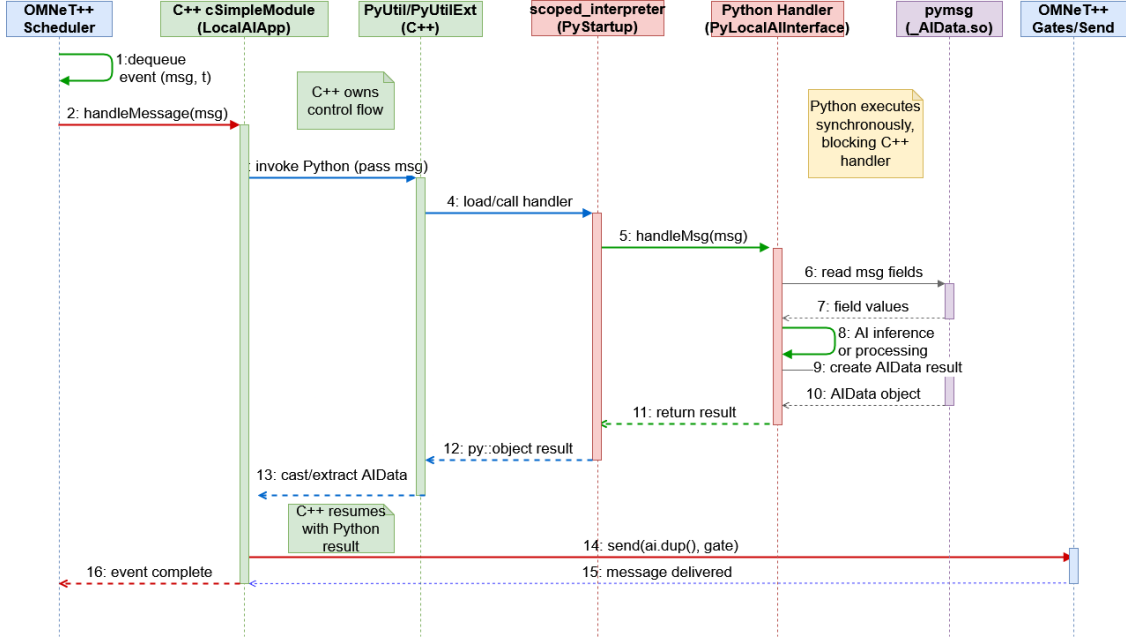


Fig. 5: PyEmbed Execution sequence.

scheduler dequeues an event, the target C++ module executes its `handleMessage()`. If it is an AI-enabled module, it invokes Python by (i) ensuring the scoped interpreter is available (`PyStart`) and (ii) calling a Python handler/callback with the OMNeT++ message exposed through bindings. Python execution blocks the C++ event handler, maintaining discrete-event semantics while allowing arbitrary Python computation. Python returns a result object (e.g., an `AIData`-type message/object), which the C++ module then extracts/casts and uses to continue normal OMNeT++ processing (e.g., send on a gate).

For integration from a user’s perspective, the workflow is as follows:

- Create Python-visible messages by adding or modifying `.msg` files under `pymsg/` for any data exchanged with Python.
- Run `MsgPyBinder.py` to automatically generate all required `pybind11` C++ bindings plus Python wrappers as part of the build.
- Write the callback/handler script in Python that reads message fields via the bindings and performs inference/processing.
- Implement or configure an `AIApps` module only when/where needed to initialize `PyStart` and invoke the Python handler within `handleMessage()`, then send the produced result back into the OMNeT++ message flow.

D. ShmGymBridge: Shared-Memory Gym Coupling for High-Frequency RL Control

While `PyEmbed` and `PyCoSim` address inference and event-driven AI, reinforcement learning requires iterative agent-environment stepping (reset/step) with Gym-compatible tooling. `ShmGymBridge` exposes OMNeT++ as a Gymnasium

environment using shared-memory IPC, avoiding socket overhead while maintaining agent-driven control flow.

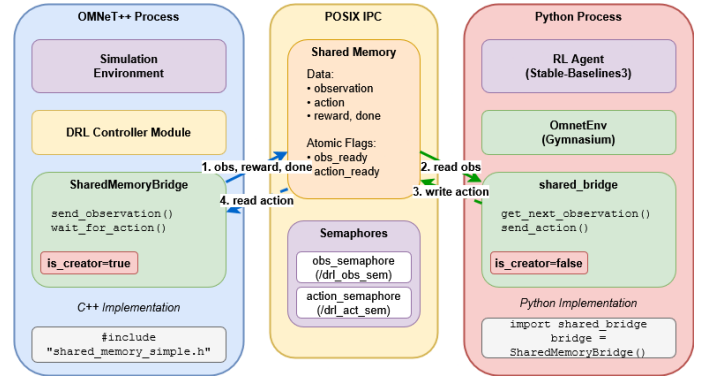


Fig. 6: ShmGymBridge Architecture overview.

Fig. 6 shows the process-level architecture of `ShmGymBridge`. The OMNeT++ process contains the simulation environment and a DRL controller module that invokes a C++ `SharedMemoryBridge` to manage the shared memory region and semaphores. The Python process contains a Gymnasium-compatible environment wrapper (`OmnetEnv`) and an RL agent (e.g., `Stable-Baselines3`) that accesses the same IPC objects via a Python shared bridge. The shared memory region stores observations, actions, rewards, done flags, and two atomic validity indicators, synchronized via semaphores (`/drl_obs_sem` and `/drl_act_sem`) for efficient blocking wait.

The synchronization protocol (Fig. 7) alternates between four states via atomic flags (`obs_ready`, `action_ready`) and semaphores. OMNeT++ writes observations/rewards and signals the Python side, which computes actions and signals back.

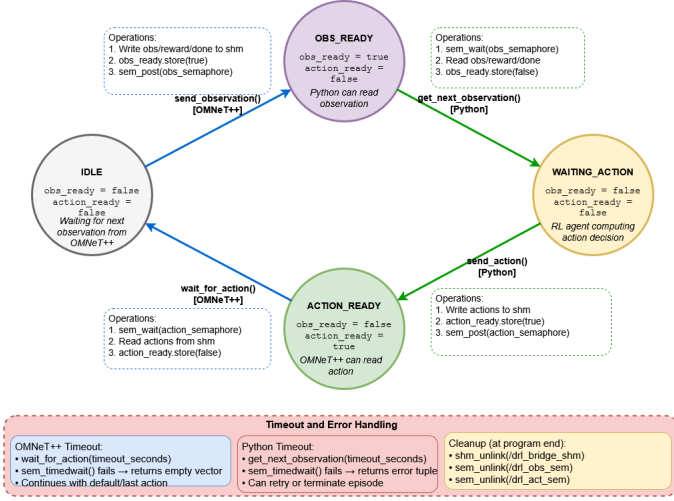


Fig. 7: ShmGymBridge state machine and protocol.

Both sides block on semaphores, ensuring efficient waiting without busy-polling.

To adapt ShmGymBridge to a new control problem, a user typically changes only the application-layer mapping while keeping the bridge unchanged:

- Define observation/action dimensions and their scaling/ranges in the shmgymsrapper.
- Define reward and termination at the simulator side and write them into the shared region alongside the observation.
- Train/infer entirely in Python using any Gymnasium-compatible RL library by instantiating the wrapper and running the chosen algorithm.

IV. PERFORMANCE CHARACTERIZATION AND INTEGRATION EXAMPLES

A. PyEmbed vs PyCoSim

To quantitatively assess the integration overhead of PyEmbed and PyCoSim, we developed a controlled benchmarking scenario that isolates C++ ↔ Python call latency under various synthetic workloads representative of typical AI-in-simulation use cases. The test network consists of two modules: *BenchPing*, which invokes Python on each message, and *BenchPong*, which immediately echoes messages back without Python involvement. This ping-pong pattern ensures that measured latency reflects only the Python integration overhead plus the synthetic workload, not network protocol complexity. We parameterized the Python handler to represent common AI workload characteristics: CPU work (simple accumulation loops: 0, 10, 100, 1000 iterations), payload manipulation (list allocation/access: 0–1000 elements), and ML inference (dense matrix multiplication using NumPy at scales $1 \times 10 \times 10$, $100 \times 100 \times 100$, $1000 \times 1000 \times 1000$). Each experiment runs 50,000–100,000 events to ensure statistical stability.

As shown in Fig. 8, PyEmbed exhibits 7.3 lower baseline overhead ($0.28 \mu s$ vs. $2.04 \mu s$), making it well-suited for high-frequency, lightweight calls such as feature extraction

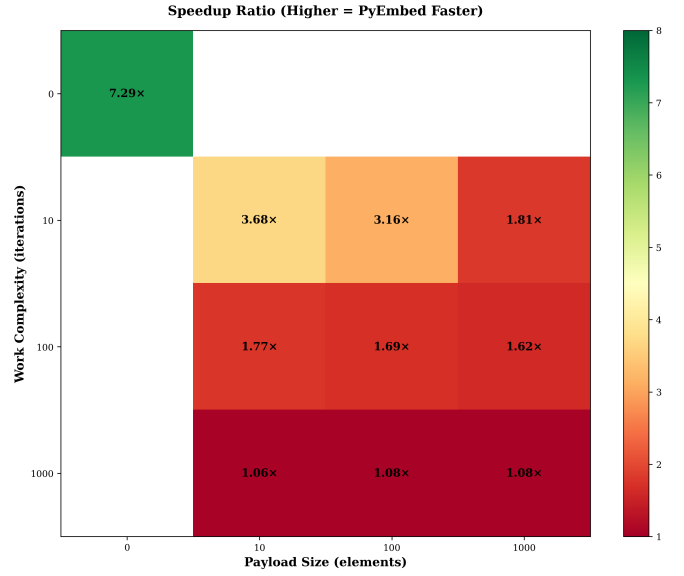


Fig. 8: PyCoSim/PyEmbed Speedup ratio comparison under different workloads.

or simple decision functions invoked at every packet arrival. This advantage stems from its minimal integration footprint: a scoped interpreter and direct pybind11 calls without the full module lifecycle machinery required by PyCoSim’s omnetpy-based approach. However, as workload complexity increases, computational cost dominates integration overhead. At heavy CPU workloads (1000 iterations) and medium-to-large matrix operations, both approaches converge to similar latencies (~ 1.0 – $1.2\times$ ratio), indicating that the choice between PyEmbed and PyCoSim becomes less critical when per-call processing exceeds $\sim 10 \mu s$.

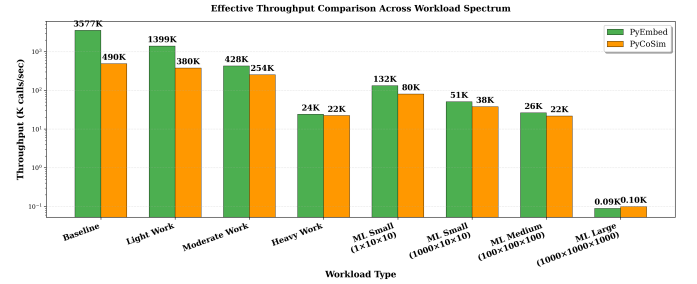


Fig. 9: PyCoSim/PyEmbed throughput comparison under different workloads.

From a throughput perspective shown in Fig. 9, PyEmbed sustains approximately 3.6 million calls/second at baseline, compared to PyCoSim’s 490k calls/second. This gap narrows for realistic AI workloads: at moderate inference complexity ($100 \times 100 \times 100$ forward pass), effective throughputs converge to $\sim 26k$ calls/second (PyEmbed) vs. $\sim 22k$ calls/second (PyCoSim). These benchmarks confirm PyOMNeT-AI’s design effectively spans the performance-flexibility spectrum, enabling users to select the appropriate pattern for their use case.

B. ShmGymBridge vs Veins-Gym

We compared ShmGymBridge to Veins-Gym across three evaluation modes: communication benchmark (no agent), agent training, and agent inference. Veins-Gym uses ZMQ with protobuf serialization, while ShmGymBridge uses POSIX shared memory with semaphore synchronization. Both approaches were evaluated with Gymnasium [9] and Stable-Baselines3, sweeping observation/action sizes and step counts.

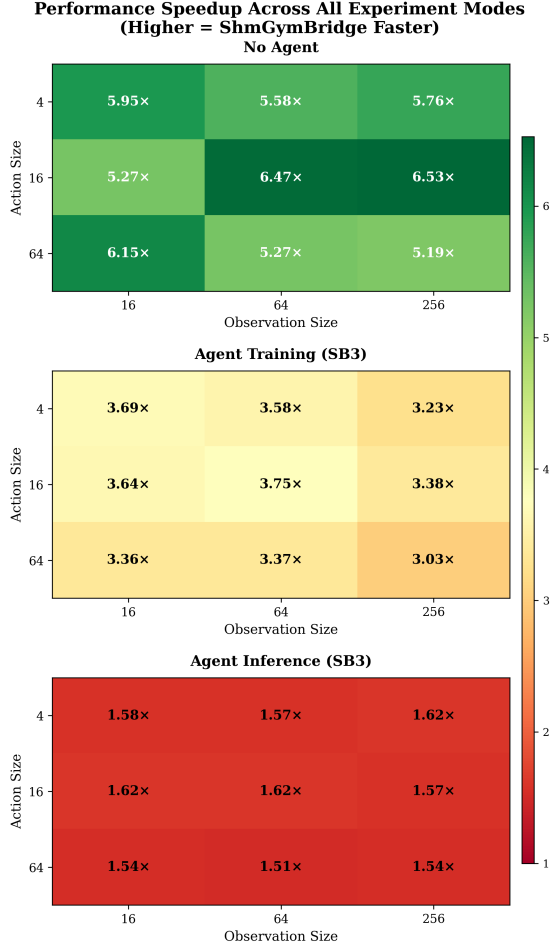


Fig. 10: Veins-Gym/ShmGymBridge speedup ratio comparison under different workloads.

The speedup heatmap in Fig. 10 plots the ratio of socket latency to shared-memory latency across observation/action sizes for all three modes. The heatmap consistently shows the largest speedups in the no-agent mode (communication-dominated), with reduced speedups in training and inference as agent computation becomes a larger portion of runtime. This confirms that shared memory reduces transport overhead across configurations, while socket/protobuf performance degrades more with payload size.

Additionally, while Veins-Gym offers a clean protocol boundary and portability across machines and operating systems, it requires coordinated builds of protobuf artifacts, complicating the build process. Any schema changes require

regenerating protobuf code and rebuilding both C++ and Python sides. The shared-memory bridge reduces runtime cost and simplifies the build pipeline since no third-party code generation is required. Therefore, shared memory is recommended for high-frequency, low-latency RL control or large parameter sweeps, while Veins-Gym remains suitable for rapid prototyping and distributed setups where modularity outweighs raw performance.

C. Integration Pattern Selection Recommendations

Based on this profiling data, the following recommendations can be made:

- For inference-only, high-frequency scenarios (e.g., per-packet classification, rapid state estimation), **PyEmbed** is recommended due to minimal overhead.
- For complex, message-driven AI components requiring rich interaction with OMNeT++ APIs (e.g., coordinating multiple simulation entities, dynamic gate management), **PyCoSim** provides expressiveness with acceptable overhead for workloads $>10 \mu\text{s}/\text{call}$.
- When training or multi-step RL is required, **ShmGym-Bridge** becomes the appropriate choice regardless of per-call latency, as it enables full integration with Gym-compatible toolchains.

D. Integration Examples

PyOMNeT-AI has been validated in multiple Simu5G-based studies, demonstrating practical applicability across diverse AI-driven networking scenarios and integration with our 3D-Network simulator [13].

In our prior work on QoS-aware scheduling [14], a DRL agent was integrated into the 5G-V2N RAN simulation [15] to influence scheduling behavior based on observed network conditions. The integration was done using PyCoSim with Tensorforce, a standalone non-Gym compliant library. This confirms that Python-native RL can be executed in lockstep with OMNeT++ simulation dynamics even for non-Gym-compliant RL frameworks, enabling learning-based control without rewriting the simulator in Python. Our ongoing work is rewriting this approach using a more optimal Gym-based integration with ShmGymBridge.

In our TN/NTN handover study, supervised learning models were incorporated into OMNeT++-based simulations to assist mobility decision-making in an integrated terrestrial/non-terrestrial setting [16]. Here, the simulation logic (protocol flow, events, and message passing) remains in C++, while Python is invoked for the learned decision function at well-defined points using PyEmbed. This reflects the common production-oriented workflow where trained models are developed in Python and then embedded for inference inside a high-fidelity simulator to evaluate system-level impact under realistic mobility/channel assumptions.

Beyond RAN optimization, PyOMNeT-AI's design patterns generalize to other OMNeT++ domains requiring AI-driven decision-making under discrete-event semantics, including IoT

network optimization, vehicular platooning control, and smart grid management.

V. CONCLUSION

Integrating Python-native AI into discrete-event network simulations requires balancing performance, expressiveness, and workflow compatibility. PyOMNeT-AI addresses this challenge through three complementary patterns—PyCoSim for message-driven AI components, PyEmbed for efficient inference, and ShmGymBridge for RL training—unified by synchronous execution semantics and low-overhead integration.

Performance characterization demonstrates PyEmbed achieves 0.28 μ s baseline overhead enabling over 3 million inference calls per second, while ShmGymBridge provides 2–6 $\times$ speedup over socket-based alternatives depending on workload and payload size. Pattern selection depends on use case requirements: PyEmbed for high-frequency inference, PyCoSim for message-driven components requiring full OMNeT++ API access, and ShmGymBridge for RL training workflows. Across these patterns, PyOMNeT-AI emphasizes explicit typed interfaces, synchronous semantics aligned with simulation time, and low-overhead integration that avoids external serialization where possible.

Current limitations include single-agent focus in ShmGymBridge, and POSIX-only shared memory implementation restricting cross-platform deployment.

While motivated by RAN optimization in Simu5G, PyOMNeT-AI’s design patterns apply broadly to domains requiring AI under discrete-event semantics: IoT networks, vehicular systems, smart grids, and industrial control systems. The framework will be released open-source alongside reference implementations, comprehensive documentation, and example scenarios.

Future work includes: (1) multi-agent ShmGymBridge with indexed shared memory regions for coordinated learning, (2) automated profiling integrated into the build system to recommend patterns based on measured call frequency and workload characteristics, (3) GPU-accelerated inference support for PyEmbed to enable larger models, and (4) cross-platform shared memory abstraction for Windows compatibility.

REFERENCES

- [1] Y. Sun, M. Peng, et al. “Application of Machine Learning in Wireless Networks: Key Techniques and Open Issues”. In: *IEEE Communications Surveys & Tutorials* 21.4 (2019).
- [2] H. Afifi, S. Pochaba, et al. “Machine Learning With Computer Networks: Techniques, Datasets, and Models”. In: *IEEE Access* 12 (2024), pp. 54673–54720.
- [3] W. Saad, M. Bennis, et al. “A Vision of 6G Wireless Systems: Applications, Trends, Technologies, and Open Research Problems”. In: *IEEE Network* 34.3 (2020).
- [4] H. Yang, A. Alphones, et al. “Artificial-Intelligence-Enabled Intelligent 6G Networks”. In: *IEEE Network* 34.6 (2020).
- [5] G. Nardini, A. Noferi, et al. “Exploiting Simu5G for generating datasets for training and testing AI models for 5G/6G network applications”. In: *SoftwareX* 21 (2023), p. 101320.
- [6] D. Stolpmann. *omnetpp-ml: Machine Learning in OM-NeT++*. GitHub repository. Repository last updated Jan 26, 2024 (per GitHub). ComNetsHH (Institute of Communication Networks, Hamburg University of Technology), 2024.
- [7] M. Schettler, D. S. Buse, et al. “How to Train your ITS? Integrating Machine Learning with Vehicular Network Simulation”. In: *2020 IEEE Vehicular Networking Conference (VNC)*. 2020, pp. 1–4.
- [8] H. Yin, P. Liu, et al. “ns3-ai: Fostering Artificial Intelligence Algorithms for Networking Research”. In: *Proceedings of the 2020 Workshop on Ns-3*. WNS3 ’20. Gaithersburg, MD, USA: Association for Computing Machinery, 2020, 57–64.
- [9] M. Towers, A. Kwiatkowski, et al. “Gymnasium: A Standard Interface for Reinforcement Learning Environments”. In: *arXiv preprint arXiv:2407.17032* (2024). Accepted at NeurIPS Datasets and Benchmarks 2025; arXiv version updated 2025-11-02.
- [10] G. Brockman, V. Cheung, et al. “OpenAI Gym”. In: *arXiv preprint arXiv:1606.01540* (2016).
- [11] P. Gawłowicz and A. Zubow. “ns-3 meets OpenAI Gym: The Playground for Machine Learning in Networking Research”. In: *22nd ACM International Conference on Modelling, Analysis and Simulation of Wireless and Mobile Systems (MSWiM 2019)*. Miami Beach, FL: ACM, Nov. 2019.
- [12] M. Modenesi. *Omnety: using Python to write OM-NeT++ simulations*. <https://github.com/mmodenesi/omnety>. 2021.
- [13] M. Roshdi, S. Roy, et al. “System Level Simulator for 3D Mobile Networks”. In: *12th Advanced Satellite Multimedia Systems Conference and the 18th Signal Processing for Space Communications Workshop (ASMS/SPSC)*. Barcelona, Spain, Feb. 2025.
- [14] E. Swistak, M. Roshdi, et al. “QoS-DRAMA: Quality of Service Aware Drl-Based Adaptive Mid-Level Resource Allocation Scheme”. In: *IEEE INFOCOM 2024 - IEEE Conference on Computer Communications Workshops (INFOCOM WKSHPS)*. 2024, pp. 1–6.
- [15] T. Deinlein, M. Roshdi, et al. “On the Impact of priority-based MAC Layer Scheduling in 5G V2N multi-application Scenarios”. In: *Proceedings of the 13th IFIP Wireless and Mobile Networking Conference, WMNC 2021*.
- [16] S. Raghunandan, M. Roshdi, et al. “Handover Management in 5G Non-Terrestrial Networks: Simulations and Testbeds”. In: *2025 28th International Workshop on Smart Antennas (WSA)*. 2025, pp. 214–220.